MACHINE LEARNING LABORATORY

Experiment 09

Support Vector Machine (SVM) for Classification

Aim

To implement Support Vector Machine (SVM) for classification using a linear kernel and an RBF kernel, and compare their performance on classification datasets.

1 Objectives

After completing this experiment, students should be able to:

1. Understand the concepts of a hyperplane, margin, and support vectors.
2. Implement a Linear SVM classifier using Python and Scikit-learn.
3. Implement an SVM classifier with an RBF kernel for non-linear decision boundaries.
4. Compare classification performance using suitable evaluation metrics.
5. Observe how kernel choice affects the decision boundary and classification results.

2 Dataset

Two datasets are used:

- **Linear dataset:** A two-class dataset generated using `make_classification`.
- **Non-linear dataset:** A two-class curved dataset generated using `make_moons`, suitable for demonstrating the RBF kernel.

The target contains two classes. Feature scaling is applied because SVM is sensitive to feature scales.

3 Key Concepts

- **Hyperplane:** The decision boundary that separates classes; in two dimensions, it is a line.
- **Margin:** The distance between the decision boundary and the nearest training points from either class.
- **Support vectors:** Training points closest to the decision boundary that influence its position.
- **Linear kernel:** Finds a linear decision boundary in the original feature space.
- **RBF kernel:** Can form non-linear decision boundaries by measuring similarity between points using a radial basis function.

4 Algorithm

Part A: Linear SVM

1. Import the required libraries.
2. Generate or load a two-class classification dataset.
3. Separate input features (X) and target labels (y).
4. Split the data into training and testing sets.
5. Scale features using `StandardScaler`, fitted on training data only.
6. Train an SVM classifier using `kernel="linear"`.
7. Predict test labels and calculate accuracy.
8. Plot the decision boundary and identify support vectors where available.

Part B: SVM with RBF Kernel

1. Generate a non-linearly separable dataset using `make_moons`.
2. Split the dataset into training and testing sets.
3. Scale features using `StandardScaler`, fitted on training data only.
4. Train an SVM classifier using `kernel="rbf"`.
5. Predict test labels and calculate accuracy.
6. Plot the non-linear decision boundary.
7. Compare the kernel behaviour and test performance.

5 Implementation

5.1 Step 1: Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification, make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.pipeline import make_pipeline
```

5.2 Step 2: Create the Linear Dataset

```
X_linear, y_linear = make_classification(
    n_samples=300,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    class_sep=1.8,
    random_state=42
```

```
)

X_train_l, X_test_l, y_train_l, y_test_l = train_test_split(
    X_linear, y_linear, test_size=0.25,
    random_state=42, stratify=y_linear
)
```

The dataset has two input features and a binary target. Its class separation makes a linear decision boundary easier to observe.

5.3 Step 3: Train and Evaluate Linear SVM

```
linear_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel="linear", C=1.0)
)
linear_svm.fit(X_train_l, y_train_l)
y_pred_l = linear_svm.predict(X_test_l)

print("Linear SVM Accuracy:",
      accuracy_score(y_test_l, y_pred_l))
print(classification_report(y_test_l, y_pred_l))
```

The pipeline scales the features before training the SVM. The scaler is fitted only on training data.

5.4 Step 4: Plot a Decision Boundary

```
def plot_decision_boundary(model, X, y, title):
    x_min, x_max = X[:, 0].min() - 0.8, X[:, 0].max() + 0.8
    y_min, y_max = X[:, 1].min() - 0.8, X[:, 1].max() + 0.8

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 300),
        np.linspace(y_min, y_max, 300)
    )
    grid = np.c_[xx.ravel(), yy.ravel()]
    Z = model.predict(grid).reshape(xx.shape)

    plt.figure(figsize=(7, 5))
    plt.contourf(xx, yy, Z, alpha=0.2, cmap="coolwarm")
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors="k",
                cmap="coolwarm", s=25)
    plt.title(title)
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()

plot_decision_boundary(
    linear_svm, X_test_l, y_test_l,
    "Linear SVM Decision Boundary"
)
```

The plot displays the regions predicted for each class. With a linear kernel, the boundary is a straight line in two dimensions.

5.5 Step 5: Create a Non-Linear Dataset

```
X_moons, y_moons = make_moons(
    n_samples=300, noise=0.20, random_state=42
)
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_moons, y_moons, test_size=0.25,
    random_state=42, stratify=y_moons
)
```

The `make_moons` dataset contains two curved groups that a single straight line cannot separate well.

5.6 Step 6: Train and Evaluate RBF SVM

```
rbf_svm = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0, gamma="scale")
)
rbf_svm.fit(X_train_r, y_train_r)
y_pred_r = rbf_svm.predict(X_test_r)

print("RBF SVM Accuracy:",
      accuracy_score(y_test_r, y_pred_r))
print(classification_report(y_test_r, y_pred_r))
```

5.7 Step 7: Plot the RBF Decision Boundary

```
plot_decision_boundary(
    rbf_svm, X_test_r, y_test_r,
    "SVM with RBF Kernel: Non-Linear Boundary"
)
```

The RBF kernel can create a curved boundary that follows the shape of non-linear data more effectively than a straight line.

5.8 Step 8: Compare the Models

```
print("Linear SVM Accuracy:",
      accuracy_score(y_test_l, y_pred_l))
print("RBF SVM Accuracy:",
      accuracy_score(y_test_r, y_pred_r))
```

Note: These scores come from different generated datasets, so they are not a strict comparison of the kernels. For a fair comparison, train both kernels on the same dataset and use the same train-test split.

6 Observations

Complete the table after running the experiment.

Observation	Result
Number of observations in the linear dataset	
Number of input features	
Linear SVM test accuracy	
Shape of the linear decision boundary	
RBF SVM test accuracy	
Shape of the RBF decision boundary	
What are support vectors?	
Which kernel is more suitable for curved data?	

7 Result

Linear SVM and RBF-kernel SVM classifiers were implemented using Scikit-learn. Their decision boundaries and test performance were observed and compared.

8 Viva Questions

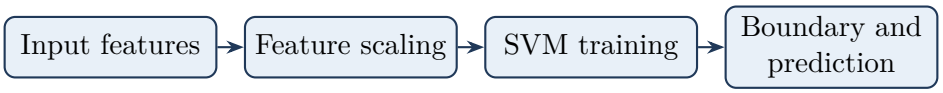
1. What is a Support Vector Machine?
2. What is a hyperplane?
3. What is the margin in SVM?
4. Why are certain training points called support vectors?
5. What is the difference between a linear kernel and an RBF kernel?
6. Why is feature scaling important for SVM?
7. What is the role of parameter C?
8. What does `gamma` control in an RBF kernel?
9. Why might linear SVM perform poorly on curved data?
10. Why should the scaler be fitted only on training data?
11. Why should evaluation use unseen test data?
12. Does high training accuracy always mean a model generalizes well?

9 Student Tasks

1. Execute the Linear SVM implementation and plot its decision boundary.
2. Execute the RBF SVM implementation and plot its decision boundary.
3. Record test accuracy and the classification report for each model.
4. Change C to 0.1, 1, and 10 for the RBF model and compare the results.
5. Try `gamma="scale"` and `gamma=0.1`; observe changes in the boundary.

6. Train both kernels on the same dataset and same train-test split, then compare test performance.
7. Write two observations about when a linear kernel or an RBF kernel is more suitable.

10 Important Learning Point



SVM seeks a decision boundary with a large margin. A linear kernel creates a linear boundary, whereas an RBF kernel can model non-linear boundaries.